

AirGuard

Implementation Guide — Stages L5 to L7

How we take the finished feature pipeline all the way to a live, self-refreshing forecast app. Written for Nourseen — where we are, what each remaining stage does, how to build it, and the traps to avoid.

Project NTI × Huawei AI Capstone	Team Mohamed Mahmoud & Nourseen Saeed	Deadline 7 Sep 2026, 09:00
--	--	--------------------------------------

Where we are now (L1–L4 done)

The whole data-engineering half is built, tested (20/20 passing), and pushed. The model layer can start immediately — the data is ready.

- **L1 Ingest** — `src/ingest.py` pulls hourly air-quality + weather from Open-Meteo (keyless).
- **L2 Store** — `src/storage.py` + Postgres, idempotent upserts (safe to re-run).
- **L3 Clean + EDA** — `src/clean.py` + notebooks/`eda.ipynb`; data is gap-free.
- **L4 Features** — `src/features.py` builds the features table: **8,748 rows, 37 columns, 0 nulls**.

The one number that governs L5 — the persistence baseline

PM2.5 barely changes hour-to-hour, so the naive rule "next value = current value" is a strong baseline. From the EDA (Cairo 2024): MAE ~ **2.2 (1h)**, 7.9 (6h), 9.4 (12h), 6.6 (24h). Our model **MUST** beat this. 1h is nearly unbeatable; 24h is easy for the naive rule again (daily cycle). **Target horizon = 6–12h** — that is where ML has real room to win.

L5 — Model (`src/train.py` + `src/predict.py`)

Goal: read the feature table, train a model that forecasts PM2.5 6–12h ahead, prove it beats the persistence baseline, and version it in MLflow.

Step 1 — Add the ML dependencies

```
# add to requirements.txt, then: pip install -r requirements.txt
scikit-learn
xgboost
lazypredict
mlflow
```

Step 2 — Load features and split by TIME (never randomly)

Random splits leak the future into training on time-series data and give a fake-good score. Train on the earlier months, test on the later ones.

```

from src.storage import read_table
from src.features import feature_target_split

df = read_table("features", city="Cairo")           # already clean, no nulls
X, y = feature_target_split(df, horizon=6)         # 6h target; also try 12h

split = int(len(df) * 0.8)                         # first 80% = train
X_train, X_test = X.iloc[:split], X.iloc[split:]
y_train, y_test = y.iloc[:split], y.iloc[split:]

```

Step 3 — Shortlist with LazyPredict, then commit to XGBoost

LazyPredict fits ~40 models fast so our choice is evidence-based, not a guess. Expect gradient-boosted trees on top for tabular data — then tune XGBoost properly.

```

from lazypredict.Supervised import LazyRegressor
reg = LazyRegressor()
models, _ = reg.fit(X_train, X_test, y_train, y_test)
print(models.head())          # screenshot this table for the slides

from xgboost import XGBRegressor
model = XGBRegressor(n_estimators=400, max_depth=6,
                    learning_rate=0.05, subsample=0.9)
model.fit(X_train, y_train)

```

Step 4 — Evaluate AGAINST the baseline (the real test)

Report improvement over persistence, not just R^2 . A pretty R^2 of 0.95 that barely beats the naive rule is not a win — say so honestly.

```

import numpy as np
from sklearn.metrics import mean_absolute_error, root_mean_squared_error

pred = model.predict(X_test)
mae = mean_absolute_error(y_test, pred)

# same-horizon persistence baseline on the test set
baseline = X_test["pm2_5"].to_numpy()          # current value = naive guess
base_mae = mean_absolute_error(y_test, baseline)

print(f"model MAE {mae:.2f} vs baseline MAE {base_mae:.2f}")
print(f"improvement: {(base_mae - mae) / base_mae * 100:.1f}%")

```

Step 5 — Log everything to MLflow

```

import mlflow, mlflow.xgboost
with mlflow.start_run(run_name="xgb_6h"):
    mlflow.log_params({"horizon": 6, "max_depth": 6, "n_estimators": 400})
    mlflow.log_metrics({"mae": mae, "baseline_mae": base_mae})
    mlflow.xgboost.log_model(model, "model")      # versioned artifact for L6

```

Step 6 (optional) — the health-category classifier

If we keep a classifier, it must predict the FUTURE hour's category, not the current one (else it is circular — it would just reverse the PM2.5-to-category formula). Classes are imbalanced (~80% Moderate), so use class weights and judge on **F1 / recall**, not accuracy.

L5 traps

(1) No random split. (2) No target column left in X — `feature_target_split` already strips them. (3) Lead with improvement-over-baseline. (4) Save the model via MLflow so L6 loads the exact same version.

Deliverables: `src/train.py`, `src/predict.py` (load model, forecast, map to advisory), MLflow runs, the LazyPredict table for the slides.

L6 — Serving (app/streamlit_app.py)

Goal: a Streamlit page that shows the next-N-hour PM2.5 forecast, the plain-language advisory, and a Refresh button for the live demo. This is what the examiners actually see — keep it clean and reliable.

The flow

- Load the trained model from MLflow (the version L5 logged).
- Pull the latest feature row from Postgres (same columns the model trained on).
- Predict PM2.5 for t+N, map it to a category, show a coloured card.
- **Refresh button** re-ingests the last few hours (reuse L1+L2), rebuild features (L4), predict again. This is the live loop for the demo.

```
import streamlit as st, mlflow.xgboost
from src.storage import read_table
from src.features import feature_target_split
from src.predict import to_advisory          # PM2.5 -> Good/Moderate/Unhealthy/...

st.title("AirGuard – Cairo PM2.5 forecast")
model = mlflow.xgboost.load_model("models:/airguard_xgb/latest")

if st.button("Refresh with latest data"):
    from dags.hourly_ingest import run_once    # reuse the ingest+store+features path
    run_once()

df = read_table("features", city="Cairo").tail(1)
X, _ = feature_target_split(df, horizon=6)
pm = float(model.predict(X)[0])
cat, advice = to_advisory(pm)

st.metric("PM2.5 in 6 hours", f"{pm:.1f} µg/m³", cat)
st.info(advice)
```

L6 traps

The feature columns at serving time must match training EXACTLY (same names, same order) — that is why L6 reuses `feature_target_split` instead of hand-building the row. Show times in Cairo local time even though we store UTC. Resist adding fancy charts until the core forecast + advisory works.

Deliverables: `app/streamlit_app.py`, a `to_advisory()` helper in `src/predict.py`, plus `streamlit` in requirements.

L7 — Monitoring & retrain loop (dags/)

Goal: the pipeline keeps itself fresh — new data flows in on a schedule, the model is checked for drift/error, and it retrains on a recent window when it slips. This is the "Ops" in MLOps and scores the deployment marks.

Two scheduled jobs

- **Hourly ingest** — fetch the last few hours (L1), upsert (L2), rebuild features (L4). Idempotent, so overlaps are harmless.
- **Periodic retrain** — refit on a recent window, compare the new MAE to the baseline and to the current model; if better, log a new MLflow version and let L6 pick up `latest`.

```
# dags/hourly_ingest.py – importable so Airflow OR a plain scheduler can call it
def run_once():
    from datetime import date, timedelta
    from src.ingest import fetch_air_quality, fetch_weather
    from src.storage import upsert_dataframe
    from src.clean import clean_dataframe
    from src.features import build_features
    start = str(date.today() - timedelta(days=2)); end = str(date.today())
    air = clean_dataframe(fetch_air_quality(start, end))
    wx = clean_dataframe(fetch_weather(start, end))
    upsert_dataframe(air, "raw_air_pollution")
    upsert_dataframe(wx, "raw_weather")
    upsert_dataframe(build_features(air, wx), "features")
```

L7 trap — Airflow-in-Docker eats time

Airflow setup routinely breaks for beginners and can sink the deadline. Because every step above is a plain importable function, the FALLBACK is trivial: schedule `run_once()` with cron or APScheduler and still demo a real automated loop. Ship the working fallback first; attempt Airflow only if time remains.

Deliverables: `dags/hourly_ingest.py`, `dags/retrain.py` (or an APScheduler script), a short drift/error check, and a note in the README on how the loop runs.

Suggested order & who does what

Both of us are MLOps engineers — pair on the model and the demo, split the rest so we don't collide.

Stage	Lead	Output
L5 model	Pair (Mohamed drives training, Nourseen eval + MLflow)	<code>train.py</code> , <code>predict.py</code> , MLflow runs
L6 Streamlit	Pair (this is the demo — build it together)	<code>streamlit_app.py</code> , <code>to_advisory()</code>
L7 ingest DAG + scheduler	Mohamed	<code>dags/hourly_ingest.py</code> + fallback
L7 retrain + drift check	Nourseen	<code>dags/retrain.py</code> , monitor note
README, slides, LinkedIn	Pair	public repo + presentation

Definition of done (rubric-aligned)

- Model beats the persistence baseline at 6–12h, improvement stated in %.
- MLflow shows versioned runs with metrics.
- Streamlit app forecasts + advises + refreshes live.
- Automated loop runs (Airflow or scheduler fallback).
- Public GitHub repo + README + slides + LinkedIn post.

Verify any layer yourself with the commands in the Obsidian *Run-and-Verify* note. Questions? Ping me. — Mohamed